

Angular Forms

Two ways to build the same sign-up form: **Template-Driven** keeps logic in the HTML, while **Reactive** defines the form and validation in the TypeScript class.

1 · TWO APPROACHES

Same form, two different philosophies

TEMPLATE-DRIVEN	REACTIVE
Most logic lives in the HTML, using <code>ngModel</code>	Form structure & validation defined in the TS class
Template does more of the work	Template just displays what the class defines

2 · TEMPLATE-DRIVEN FORMS

Setup, template, and validation

Component

```
import { FormsModule } from '@angular/forms';

@Component({ selector: 'app-signup', imports: [FormsModule] })
export class SignupComponent {
  name = '';
  email = '';
  onSubmit() { console.log(this.name, this.email); }
}
```

Template with ngModel

```
<form #signupForm="ngForm" (ngSubmit)="onSubmit()">
  <input name="name" [(ngModel)]="name" required minlength="3" #nameField="ngModel">
  <input name="email" [(ngModel)]="email" required email #emailField="ngModel">
  <button type="submit" [disabled]="signupForm.invalid">Sign Up</button>
</form>

@if (nameField.invalid && nameField.touched) {
  <small>Name is required / at least 3 characters.</small>
}
```

`[(ngModel)]` is **two-way binding**. **touched** means the user clicked in and left the field — always check `invalid && touched` together.

Setup, template, and validation

Component

```
import { FormGroup, FormControl, Validators } from '@angular/forms';

signupForm = new FormGroup({
  name: new FormControl('', [Validators.required, Validators.minLength(3)]),
  email: new FormControl('', [Validators.required, Validators.email]),
});
onSubmit() { console.log(this.signupForm.value); }
```

Template with formGroup

```
<form [formGroup]="signupForm" (ngSubmit)="onSubmit()">
  <input formControlName="name">
  <input formControlName="email">
  <button type="submit" [disabled]="signupForm.invalid">Sign Up</button>
</form>
```

`FormControl` = one field's value + rules. `FormGroup` bundles controls together. Because the whole form is a plain object in the class, it's easy to unit test — no HTML needed.

4 · DECISION GUIDE

When to use which?

CHOOSE TEMPLATE-DRIVEN WHEN...	CHOOSE REACTIVE WHEN...
The form is simple (few fields)	The form is complex or has many fields
You want the fastest way to get something working	Validation rules change dynamically
Validation rules are straightforward	You need to unit test form logic easily
	You're building a larger, production app

5 · BEST PRACTICES

- Always disable the submit button while the form is invalid: `[disabled]="form.invalid"`.
- Only show error messages after the field has been touched.
- Keep error messages specific ("Name must be at least 3 characters") rather than generic.
- Don't mix Template-Driven and Reactive approaches in the same form — pick one and stay consistent.